

Full-Text Search Engine on a Suffix-Array Index

Project Details

Submitted By :

M.Sai Chaitanya 2520090023

D.Bala Charan 2520090087



Motivation & Problem Statement

The Challenge

Modern applications need **fast search across large text collections**. Exact keyword search is useful, but it does not handle **substring and phrase queries** well.

Linear scans are too slow. Inverted indexes are efficient for keywords, while suffix trees support substrings but use more memory and are more complex.

The Proposed Solution

A **suffix array** provides a compact index of all suffixes. With an **LCP array**, it supports faster binary search and efficient substring matching with lower memory use.

Key Objectives

- Enable fast search over large document collections
- Support substring, prefix, and phrase queries
- Build the suffix array in **$O(n \log^2 n)$** time
- Use Kasai's Algorithm for LCP construction
- Speed up queries with LCP-based search
- Rank results by relevance
- Keep memory usage lower than suffix trees

System Capabilities & Comparative Advantage

Core Technical Capabilities

Fast Full-Text Searching

Sorted suffix arrays enable efficient search over large text collections.

Flexible Query Types

Supports **substring**, **prefix**, and **phrase** queries.

Efficient Index Construction

Suffix arrays build in $O(n \log^2 n)$; Kasai's Algorithm builds LCP in $O(n)$.

Relevance-Based Ranking

Results are ranked by term frequency and match quality.

Comparative Advantages

vs. Suffix Trees

Lower memory use with no pointer-heavy tree structure.

vs. Inverted Indexes

Arbitrary substring support without tokenization.

vs. Linear Search

Faster queries via binary search on sorted suffixes.

Target Application Domains

 **Digital Libraries**

 **Bioinformatics**

 **Search Engines**

 **Text Analytics**

Software Requirements & Technologies Used

Software Requirements

Operating System	Windows / Linux
Programming Language	Java
IDE	VS Code / IntelliJ IDEA
Runtime	Java Runtime Environment (JRE)
Input Format	Text documents / corpus
Output Format	Ranked search results
Memory	4 GB minimum
Storage	1 GB+

Core Algorithms & Techniques

Suffix Array

Sorted suffix positions for efficient pattern matching via binary search.

LCP Array + Kasai's Algorithm

Builds longest common prefixes in $O(n)$ and reduces repeated character comparisons.

LCP-Accelerated Binary Search

Uses LCP information to narrow comparisons during search.

Text Preprocessing & Ranking

Normalizes text and ranks matches by frequency and relevance.

Comparative Analysis of Search Approaches

The table compares key search strategies and highlights why the proposed system is chosen.

Approach	Main Idea	Advantages	Limitations
Linear Search	Scans text for each query	Simple; no index needed	Slow on large data
Inverted Index	Maps tokens to documents	Fast keyword search	Poor for substring queries
Suffix Tree	Trie of all suffixes	Fast, supports all substrings	Very high memory use
Suffix Array	Sorted suffix positions	Lower memory; efficient search	Redundant comparisons without LCP
Proposed System	Suffix Array + LCP	Fast substring and phrase search	Higher build cost; best at scale

System Pipeline & Expected Outcomes

7-Step Implementation Pipeline

- 1** Text Collection
Gather target documents or corpus data for the application.
- 2** Text Preprocessing
Clean, normalize, and tokenize input for indexing.
- 3** Suffix Array Construction
Generate and sort suffixes, then store starting positions. **$O(n \log^2 n)$** .
- 4** LCP Array Construction
Use **Kasai's Algorithm** to compute adjacent suffix prefixes in **$O(n)$** .
- 5** Result Ranking
Rank matches by relevance and term frequency.
- 6** Display Search Results
Return ranked matches with positions and scores.

Expected Outcomes

⚡ Faster Search
Much faster than linear scan on large corpora.

🔍 Flexible Matching
Supports substring and phrase queries effectively.

💾 Reduced Memory
Uses less memory than suffix trees.

📈 Competitive Speed
Performs well for supported query types.